**Question 1: General (20 points)**

1. Assume in a computer system the processes fall into three priority levels, thus three ready queues. Write the steps of an aging algorithm that prevents starvation to happen in this system. (4 points)

2. Assume two processes P0 and P1, each execute a critical section CS and need to use two semaphores S1 and S2 to implement mutual exclusion over the CS. Complete the following algorithms so that deadlock will be impossible. (4 points)

Initial values for S1 and S2:

S1=...;.....;  S2=.....;

Alg for P0:

```
...
wait (.....);
wait(.....);

CS;

signal(.....)
signal(.....)
...
```

Alg for P1:

```
...
wait (.....);
wait (.....);

CS;

signal (.....);
signal (.....);
...
```

3. Complete the following table to indicate the type of fragmentation and the reason of happening due to different memory management methods (6 points)

| Memory Management Method | Internal fragmentation (Yes/No) | External fragmentation (Yes/No) | Reason |
|---|---|---|---|
| Fixed size memory partitioning | | | |
| Variable size memory partitioning | | | |
| Paging | | | |

**Question 3: Deadlock (20 points)**

1. Show a **deadlock** and **no deadlock** scenarios involving **all processes** and **resources** using appropriate request and assignment arrows. Note that an arrow away from process shows **request**, towards it shows assignment. Suppose there are 3 processes (P0, P1, P2) and 3 resources (R0, R1, R2). Represent process by a circle, resource by a square as:-

a) A digraph for **deadlock** case: (3 points)

b) A digraph for **no deadlock** case: (3 points)

2. The resources requested by each process can be granted in increasing order of resource identification number to prevent deadlocks. Assuming in the above configuration, each process needs exactly any two resources. Draw an example graph showing a deadlock free case. Hint: if a process has requested or allocated R2, it cannot request R1. (5 points)

**Question 2: Process scheduling (20 points)**

The following table shows the details of a four concurrent process case without I/O.

| Process | Arrival Time | CPU burst Time |
|---|---|---|
| 1 | 0 | 20 |
| 2 | 5 | 10 |
| 3 | 5 | 5 |
| 4 | 10 | 10 |

a) Schedule the processes using Gantt chart showing the execution sequences according to the given algorithms (12 points).

**FCFS-First Come First Served:**

**SJF-Shortest Job First without pre-emption:**

**RR-Round Robbin with a period of 10 time units:**

b) Compute average wait times for the shown algorithms: (8 points)

**FCFS-First Come First Served:**

**SJF-Shortest Job First:**

**RR-Round Robbin with a period of 10 time units:**

3. Use Banker's algorithm for the following process (P0, P1, P2) and resource (A,B,C) data:

| Process | Max Resource Need | | | Resource Allocated | | | Avail (Remained Resource) | | |
|---|---|---|---|---|---|---|---|---|---|
| | A | B | C | A | B | C | A | B | C |
| P0 | 8 | 4 | 3 | 2 | 1 | 1 | 4 | 3 | 2 |
| P1 | 3 | 2 | 2 | 1 | 1 | 0 | | | |
| P2 | 6 | 1 | 2 | 1 | 0 | 2 | | | |

a) Compute the Need matrix: (3 points)

| | A | B | C |
|---|---|---|---|
| P0 | ---- | ---- | ---- |
| P1 | ---- | ---- | ---- |
| P2 | ---- | ---- | ---- |

b) Compute the state of the system to show if there is a SAFE process execution sequence. Otherwise, the system would be UNSAFE. Each time a process completed its execution, show the updated available resources. (6 points)

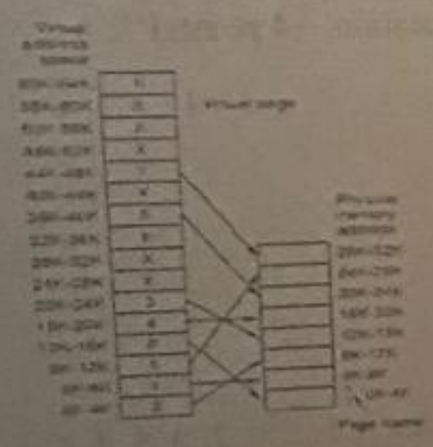
[illegible] (N) virtual memory uses cache memory, main memory (RAM) and hard disk to handle memory [illegible] memory access time (EAT). If the cache hit rate is 70 nanoseconds (ns), 100 ns, respectively, hard-disk page swap-out or swap-in times is 10 milliseconds (msec). (8 points)

[illegible] memory access time (EAT). If the cache hit rate is 80%. Formulate computation of EAT [illegible] the numeric value.

[illegible] memory access time (EAT), if the cache hit rate is 80% and main memory hit rate (after [illegible] 80%. Formulate computation of EAT first, and then compute the numeric value. Assume [illegible] page situation.

## Question 4: Memory management (20 points)

1. Given the memory mapping below, find the physical address corresponding to each of the following virtual addresses: *Hint: pages and frames are numbered starting from 0 each.* (4 points)



(a) 022A0 .....

b) 0B220 : .....

2. If at this time, 12 different virtual pages are referenced what would be the maximum number of page faults? Hint: note that 8 pages are already in the main memory (3 points)

3. In the above memory configuration, if the virtual addressing uses two level page table, instead of one, what is the total number of first and second level virtual table entries. *Hint: note that the total number the virtual entries in one level page table is 16.* (4 points)

4. A computer has four pages in the memory, whose loading time, last reference time and the R and M bits are as shown below. A page fault activates the page replacement algorithm. (9 points)

| Page# | loaded | referenced | R | M |
|---|---|---|---|---|
| 0 | 100 | 280 | 1 | 0 |
| 1 | 200 | 240 | 0 | 1 |
| 2 | 150 | 250 | 0 | 0 |
| 3 | 250 | 300 | 1 | 1 |

Based on this data write the page replaced by each algorithm below:

(a) FIFO, according to load time:

(b) LRU-Least recently Used, according to reference time:

(c) Second chance, according to R and M bits:

## Question 5: Memory Management: (20 points)

1. In a main memory of 8 frames, there are 3 processes and each with virtual memory of 4 pages. Assume 2 rounds of page references for the processes to complete execution. In each round all the pages of all 3 processes are referenced in order of process and page number, with no page modifications. Compute the total number of **page-faults**, showing details of the computation. (4 points)

2. Assume that during the execution of a process, the memory reference string is 1, 2, 0, 2, 0, 5, 4, 6, 5, 1, 2, 4, 0, 1, 3, 2, in terms of virtual page numbers is as follows. Take the size of the virtual memory as three (3) frames only. Based on stack based page replacement principle:-
   - Update the memory each time there is a page reference based on LRU-Least Recently Used method. (8 points)
   - Mark if there is a page fault, in the bottom second row, using least recently used-**LRU** page replacement algorithms. (4 points)
   - Enter the distance for each page reference, relative to the first memory frame. (4 points)

| Reference order # | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Referenced page | 1 | 2 | 0 | 2 | 0 | 5 | 4 | 6 | 5 | 1 | 2 | 4 | 0 | 1 | 3 | 2 |
| Main memory content | | | | | | | | | | | | | | | | |
| | | | | | | | | | | | | | | | | |
| | | | | | | | | | | | | | | | | |
| Out of memory | | | | | | | | | | | | | | | | |
| | | | | | | | | | | | | | | | | |
| | | | | | | | | | | | | | | | | |
| Enter P if page-fault | | | | | | | | | | | | | | | | |
| Distance from the first memory page | | | | | | | | | | | | | | | | |